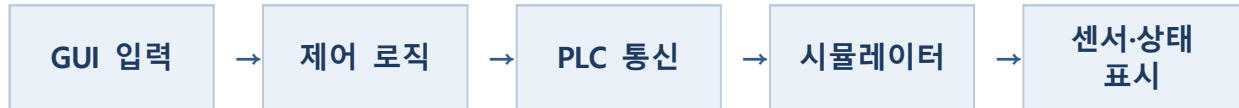


교과목명		융합UI실습			
학 번	2601110288	이 름	박수용		
능력단위	인터페이스 구현 (2001020212_19V5)	능력단위 요소	인터페이스 설계서 확인하기	평가방법	평가자 체크리스트
			인터페이스 기능 구현하기		
			인터페이스 구현 검증하기		
			인공지능 모델 학습하기		
요구사항					
[실습6] 리프트제어를 기반으로 자동 운전시 전체 시퀀스가 자동으로 진행되며, 수동 조작시 각 실린더와 리프트를 각각 제어 가능해야 함.					
[요구사항] 자동운전이 온전히 되어야 함.					
[요구사항] 수동조작이 각 컴포넌트 별 가능해야 함.					
[요구사항] GUI에 현재의 상태를 모두 표시하여야 함.					
[요구사항] 보고서는 시나리오별 동작사항에 관하여 검증되어야 함.					
[요구사항] 보고서는 컴포넌트별 핵심 코드를 첨부하여 설명되어야 함.					
[요구사항] 보고서는 PDF로 제출되어야 함.					
[동작정보]					
X 디바이스(센서데이터) B실린더 후진 : X03 B실린더 전진 : X02 C실린더 후진 : X04 C실린더 전진 : X05 리프트센서A : XA 리프트센서B : XB			Y 디바이스(출력데이터) B실린더 전진 : Y01 B실린더 후진 : Y02 C실린더 전진 : Y03 C실린더 후진 : Y04 LiftA Up : Y5 LiftA Down : Y6 LiftB Up : Y8 LiftB_Down : Y7		
평가문항(수행내용)					
1. 과제 목적					
이번 과제의 목표는 C# Windows Forms와 PLC 시뮬레이터를 연결하여 컨베이어 위의 물체를 두 개의 실린더와 두 개의 리프트로 순환시키는 제어 프로그램을 구현하는 것이었습니다. 단순히 장치를 움직이는 것에 그치지 않고, 자동 운전과 수동 조작을 구분하고 현재 센서 상태를 화면에서 확인할 수 있도록 구성하였습니다.					
프로그램은 사용자가 연결 버튼을 눌러 PLC와 통신을 시작한 뒤, 자동 운전에서는 센서 조건에 따라 다음 동작으로 넘어가며 전체 공정을 반복합니다. 수동 모드에서는 B.C실린더와 리프트A.B를 버튼별로 직접 조작할 수 있습니다. 또한 자동 운전 중에는 수동 버튼을 비활성화하여 서로 다른 명령이 동시에 입력되지 않도록 하였습니다.					

2. 시스템 동작 구조

전체 프로그램은 사용자 입력, 제어 로직, PLC 통신, 센서 확인, 화면 갱신의 순서로 동작합니다. 자동 운전에서는 센서 값이 상태 머신의 다음 단계 조건으로 사용되고, 수동 조작에서는 버튼 입력이 바로 출력 비트 변경으로 연결됩니다.



3. GUI 구성

GUI는 연결과 운전 제어를 담당하는 상단 영역, 각 장치를 직접 움직이는 수동 조작 영역, 현재 상태를 보여 주는 라벨 영역으로 나누었습니다. 연결 전에 운전 버튼을 비활성화하고 자동 운전 중에는 수동 버튼을 잠가 잘못된 조작을 줄였습니다.

구성 요소	구성	설계 의도
운전 버튼	연결 / 자동시작 / 자동정지	운전 흐름을 한곳에서 제어
수동 버튼 8개	B·C실린더 전진/후진, 리프트A·B 상승/하강	컴포넌트별 개별 점검 가능
상태 라벨 7개	모드·자동 단계, 센서 A·B, 실린더와 리프트 상태	현재 상태를 글자로 즉시 확인
타이머	250ms 주기	센서 읽기, GUI 갱신, 자동 시퀀스 진행

4. 설계 및 구현

4.1 전체 동작 흐름

자동 운전은 물체가 리프트B로 이동한 뒤 아래쪽으로 내려가고, C실린더가 물체를 리프트A로 밀어 올린 다음 다시 처음 위치로 돌아오는 순환 구조입니다. 각 장치의 완료 센서를 확인한 뒤 다음 명령을 보내기 때문에, 정해진 시간만 기다리는 방식보다 실제 장치 상태를 반영할 수 있습니다.

[자동 운전 순환 경로]

리프트B 상승 준비 → B실린더 전진 → 센서B 감지 → B실린더 후진 → 리프트B 하강 → 리프트A 하강 준비 → C실린더 전진 → 센서A 감지 → C실린더 후진 → 리프트A 상승 → 반복

4.2 자동 운전 · 상태 머신

자동 운전을 하나의 긴 조건문으로 처리하면 현재 단계와 다음 동작을 확인하기 어렵습니다. 이를 해결하기 위해 AutoStep 열거형으로 10개의 단계를 구분하였습니다. 타이머가 실행될 때마다 현재 단계에 필요한 센서를 확인하고, 조건이 충족되면 다음 단계로 넘어갑니다.

단계	현재 동작	다음 단계 조건	다음 단계
HomeB	B실린더 후진 원점	B 후진 완료	HomeC
HomeC	C실린더 후진 원점 또는 현재 위치 판단	C 후진 완료 또는 센서 상태	ReadyLiftBUp 등
ReadyLiftBUp	리프트B 상승	B 상승 완료	PushToLiftB
PushToLiftB	B실린더 전진	리프트센서 B 감지	RetractBAfterDetect
RetractBAfterDetect	B실린더 후진	B 후진 완료	MoveLiftBDown
MoveLiftBDown	리프트B 하강	B 하강 완료	ReadyLiftADown
ReadyLiftADown	리프트A 하강	A 하강 완료	PushToLiftA
PushToLiftA	C실린더 전진	리프트센서 A 감지	RetractCAfterDetect
RetractCAfterDetect	C실린더 후진	C 후진 완료	MoveLiftAUp
MoveLiftAUp	리프트A 상승	A 상승 완료	ReadyLiftBUp

4.3 핵심코드

코드	설명	역할
<pre>enum AutoStep { HomeB, HomeC, ReadyLiftBUp, PushToLiftB, RetractBAfterDetect, MoveLiftBDown, ReadyLiftADown, PushToLiftA, RetractCAfterDetect, MoveLiftAUp }</pre>	자동 운전의 각 단계를 이름으로 정의하였습니다.	현재 단계와 전환 흐름을 명확하게 구분합니다.
<pre>case AutoStep.PushToLiftB: if (liftBOn) autoStep = AutoStep.RetractBAfterDetect; else { WriteY(Y_B_FORWARD); label1.Text = "자동: B실린더 전진(센서B 감지 대기)"; } break;</pre>	센서B가 켜질 때까지 B실린더를 전진시키고, 감지되면 후진 단계로 이동합니다.	센서 조건을 기준으로 출력과 상태 전환을 처리합니다.
<pre>case AutoStep.MoveLiftAUp: if (liftAUp) autoStep = AutoStep.ReadyLiftBUp; else WriteY(Y_LIFTA_UP); break;</pre>	리프트A가 상승 완료되면 처음 순환 단계로 돌아갑니다.	자동 운전을 반복 구조로 만듭니다.
<pre>private void button10_Click(...) { autoMode = true; autoStep = AutoStep.HomeB; SetManualButtons(false); }</pre>	자동시작 시 첫 단계와 버튼 상태를 초기화합니다.	자동 운전 진입과 수동 조작 잠금을 담당합니다.

HomeC 단계에서는 시작 시점에 물체가 이미 리프트A나 리프트B에 놓여 있는 경우를 확인합니다. 현재 센서 상태에 맞는 단계로 이동하도록 하여, 반드시 완전히 초기화된 위치에서만 자동 운전을 시작해야 하는 문제를 줄였습니다.

4.4 수동 조작

수동 모드는 각 컴포넌트를 개별적으로 점검할 때 사용합니다. 전진과 후진 또는 상승과 하강 출력이 동시에 켜지면 장치 명령이 충돌할 수 있으므로, SetOutput 함수에서 한쪽 비트를 켤 때 반대쪽 비트는 반드시 끄도록 처리하였습니다.

코드	설명	역할
<pre>private void SetOutput(int onBit, int offBit) { ushort cur = (ushort)output; cur = (ushort)((cur onBit) & ~offBit); WriteY((short)cur); }</pre>	현재 출력값을 유지하면서 지정한 비트는 켜고 반대 비트는 끕니다.	서로 반대되는 출력의 동시 활성화를 방지합니다.
<pre>private void RunManualCommand(short onBit, short offBit, Label statusLabel, string text) { SetOutput(onBit, offBit); statusLabel.Text = text; }</pre>	출력 변경과 라벨 표시를 공통 함수로 묶었습니다.	8개 버튼의 중복 코드를 줄입니다.
<pre>private void SetManualButtons(bool enabled) { button1.Enabled = enabled; ... // button2 ~ button8 }</pre>	수동 버튼의 활성화 상태를 한 번에 변경합니다.	연결 전과 자동 운전 중의 오조작을 막습니다.

4.5 센서 상태 표시와 GUI 갱신

타이머는 250ms마다 X0 한 워드를 읽습니다. 읽어온 값에 비트 마스크를 적용하여 각 센서의 ON/OFF 상태를 구분한 뒤, 실린더와 리프트의 완료 상태를 라벨에 표시하였습니다. 자동 운전과 수동 조작 모두 같은 상태 표시 코드를 사용하므로, 어떤 모드에서도 현재 장치 상태를 확인할 수 있습니다.

코드	설명	역할
<pre>int s; if (!TryReadSensors(out s)) { label1.Text = "PLC 통신 대기/재시도 중"; return; }</pre>	센서 읽기에 실패하면 현재 주기를 건너뛰고 다음 주기에 다시 시도합니다.	일시적인 통신 오류로 프로그램이 종료되는 것을 방지합니다.
<pre>bool bFwdDone = (s & 0x0004) != 0; bool liftAOn = (s & 0x0400) != 0;</pre>	16비트 입력값에서 필요한 센서 비트를 분리합니다.	PLC의 묶음 데이터를 개별 상태값으로 변환합니다.

bool liftBOn = (s & 0x0800) != 0;		
label4.Text = "B실린더: " + (bFwdDone ? "전진 완료" : bBackDone ? "후진 완료" : "동작 중");	센서 상태를 사용자가 읽기 쉬운 문장으로 표시합니다.	GUI에서 현재 상태를 바로 확인할 수 있게 합니다.

4.6 통신 안정화와 종료 처리

PLC 통신은 타이머가 너무 빠르게 중복 실행되거나 프로그램 종료 중에 호출되면 오류가 발생할 수 있습니다. 이를 줄이기 위해 ticking 변수로 타이머 재진입을 막고, 직전과 같은 Y0 값은 다시 보내지 않도록 하였습니다. 프로그램을 닫을 때는 Y0 출력을 0으로 만든 뒤 통신을 종료합니다.

코드	설명	역할
<pre>if (ticking !plcConnected closing) return; ticking = true; timer1.Stop(); try { TickBody(); } finally { ticking = false; if (plcConnected && !closing) timer1.Start(); }</pre>	이전 타이머 처리가 끝나기 전에 다음 처리가 겹치지 않도록 합니다.	타이머 중복 실행과 통신 충돌을 줄입니다.
<pre>if (!force && word == lastWrittenOutput) return true;</pre>	직전 출력과 같은 값이면 통신을 생략합니다.	불필요한 쓰기 횟수를 줄여 안정성을 높입니다.
<pre>private void Form1_FormClosing(...) { closing = true; autoMode = false; timer1.Stop(); if (plcConnected) WriteY(0, true); control.Close(); }</pre>	폼을 닫기 전에 자동 모드와 출력을 정지하고 연결을 해제합니다.	종료 후 출력 신호가 남는 위험을 줄입니다.

5. 개발 과정에서 해결한 문제

이번 과제에서 가장 시간이 많이 걸린 부분은 코드를 작성하는 것보다 문서의 입출력 정보와 실제 시뮬레이터 동작을 맞추는 과정이었습니다. 또한 타이머와 PLC 통신이 동시에 실행될 때 발생하는 불안정성을 줄여야 했습니다. 아래는 구현 과정에서 확인한 문제와 해결 방법입니다.

문제	원인	해결 방법	결과
입출력 문서와 실제 동작 차이	C실린더 센서와 리프트B 출력 표기가 실제 반응과 다름	Y 출력을 하나씩 인가하고 X 센서 변화를 확인하여 매핑 수정	자동 시퀀스가 단계별로 정상 진행
타이머 통신 중복	이전 통신이 끝나기 전에 다음 Tick이 실행될 수 있음	ticking 플래그와 timer1.Stop/Start 사용	통신 오류로 인한 중단 감소
자동·수동 명령	자동 운전 중 수동	자동 시작 시 수동 버튼	운전 모드가

충돌 가능성	버튼을 누르면 출력이 쉬일 수 있음	비활성화, 정지 시 다시 활성화	명확히 분리됨
시작 위치가 일정하지 않음	물체가 중간 위치에 있으면 원점 순서만으로 진행이 어려움	HomeC에서 센서 상태를 확인하고 알맞은 단계로 이동	중간 상태에서도 시퀀스 연결 가능

5.1 문제 해결 과정에서 확인한 점

- PLC 제어에서는 문서의 입출력 정보를 기준으로 구현한 뒤, 실제 출력과 센서 반응이 일치하는지 함께 확인하는 과정이 필요하였습니다.
- 자동 시퀀스는 현재 단계와 완료 조건이 분리되어 있어야 오류가 발생한 위치를 찾기 쉬웠습니다.
- GUI 버튼의 활성화 상태도 제어 로직의 일부이며, 잘못된 입력을 사전에 막는 역할을 하였습니다.
- 빈 catch문은 프로그램 종단을 막는 데에는 도움이 되지만, 오류 원인을 기록하기 어렵다는 한계가 있었습니다.

6. 시나리오별 동작 검증

6.1 검증 기준 및 종합 결과

검증은 GUI에 표시되는 모드·센서·완료 상태와 PLC 시뮬레이터에서 실제로 움직이는 컴포넌트를 함께 관찰하는 방식으로 진행하였습니다. 조작 조건에 따른 예상 결과와 실제 확인 결과가 일치할 때 정상으로 판정하였습니다.

No.	시나리오	조작/조건	예상 결과	실제 확인 결과	판정
1	PLC 연결	[연결] 클릭	연결 성공 및 수동 버튼 활성화	수동(대기) 모드 전환, 버튼 활성화	정상
2	자동 운전 진입	[자동시작] 클릭	자동 모드 전환 및 수동 버튼 잠금	자동운전 중 표시, 수동 버튼 비활성화	정상
3	자동 전체 순환	자동 운전 유지	센서 조건에 따라 10단계 순환	B-리프트B·C-리프트A 순서로 반복	정상
4	자동 정지	[자동정지] 클릭	출력 0, 수동 모드 복귀	동작 정지 후 수동 버튼 재활성화	정상
5	수동 개별 제어	8개 수동 버튼 클릭	선택한 방향의 장치만 동작	각 실린더·리프트가 버튼대로 동작	정상
6	GUI 상태 표시	전 과정 관찰	모드·센서·완료 상태 실시간 갱신	7개 라벨이 동작에 따라 변경	정상
7	프로그램 종료	폼 닫기	출력 0 후 PLC 연결 해제	출력 초기화 후 정상 종료	정상

6.2 자동 운전 10단계 검증

자동 운전이 일부 단계만 수행되는 것이 아니라 한 주기를 끝까지 완료하고 다시 처음단계로 돌아가는지 확인하였습니다. 아래 표는 상태 머신의 각 단계별 검증 결과입니다.

순서	상태	출력 동작	완료/전환 조건	확인 결과	판정
1	HomeB	B실린더 후진	X03 후진 완료	B실린더 원점 후 HomeC 이동	정상
2	HomeC	C실린더 후진/현재 위치 판단	C실린더 후진 완료 또는 리프트센서 상태	시작 위치에 맞는 단계 선택	정상
3	ReadyLiftBUp	리프트B 상승	X08 상승 완료	받을 위치까지 상승	정상
4	PushToLiftB	B실린더 전진	XB 물체 감지	센서B 감지 후 후진 전환	정상
5	RetractBAfterDetect	B실린더 후진	X03 후진 완료	리프트B 하강 전환	정상
6	MoveLiftBDown	리프트B 하강	X09 하강 완료	리프트A 준비 전환	정상
7	ReadyLiftADown	리프트A 하강	X07 하강 완료	C실린더 전진 전환	정상
8	PushToLiftA	C실린더 전진	XA 물체 감지	센서A 감지 후 후진 전환	정상
9	RetractCAfterDetect	C실린더 후진	C실린더 후진 완료 감지	리프트A 상승 전환	정상
10	MoveLiftAUp	리프트A 상승	X06 상승 완료	ReadyLiftBUp 복귀·반복	정상

[자동 운전 온전성 확인]

10단계의 마지막 MoveLiftAUp에서 리프트A 상승 완료를 확인한 뒤 ReadyLiftBUp으로 복귀하였으며, 다음 물체 이송 주기가 다시 시작되는 것을 확인하였습니다. 따라서 단일 동작이 아니라 전체 자동 시퀀스가 반복되는 형태로 검증하였습니다.

6.3 수동 조작 8개 버튼 검증

수동 모드에서는 8개의 버튼을 하나씩 실행하여 각 컴포넌트가 지정된 방향으로 정상 동작하는지 확인하였습니다. 반대 방향 출력은 SetOutput 함수에서 동시에 해제되도록 구성하였습니다.

No.	수동 버튼	대상 출력	확인 내용	판정
1	B실린더 전진	Y1	B실린더만 전진하고 상태 라벨 변경	정상
2	B실린더 후진	Y2	B실린더만 후진하고 상태 라벨 변경	정상
3	C실린더 전진	Y3	C실린더만 전진하고 상태 라벨 변경	정상
4	C실린더 후진	Y4	C실린더만 후진하고 상태 라벨 변경	정상
5	리프트A 상승	Y5	리프트A만 상승하고 상태 라벨 변경	정상
6	리프트A 하강	Y6	리프트A만 하강하고 상태 라벨 변경	정상
7	리프트B 상승	리프트B 상승 출력	리프트B만 상승하고 상태 라벨 변경	정상
8	리프트B 하강	리프트B 하강 출력	리프트B만 하강하고 상태 라벨 변경	정상

6.4 GUI 현재 상태 표시 검증

표시 항목	GUI 라벨 내용	갱신 근거	결과
운전 모드/자동 단계	대기·수동·자동 및 현재 자동 단계	label1, autoStep	표시
리프트센서 A	ON(감지)/OFF	XA 비트	표시
리프트센서 B	ON(감지)/OFF	XB 비트	표시
B실린더	전진 완료/후진 완료/동작 중	X02·X03	표시
C실린더	전진 완료/후진 완료/동작 중	X04·X05 입력 비트	표시
리프트A	상승 완료/하강 완료/동작 중	X06·X07	표시
리프트B	상승 완료/하강 완료/동작 중	X08·X09	표시

[화면 해석 시 주의점]

수동 화면에서 "PLC 통신 대기/재시도 중" 문구가 잠시 표시되는 경우가 있었습니다. 이는 한 번의 센서 읽기가 실패했을 때 표시되는 안내이며, 다음 타이머 주기에서 통신을 다시 시도하도록 구현하였습니다.

6.5 실행 화면

1) PLC 연결 후 수동 대기 상태

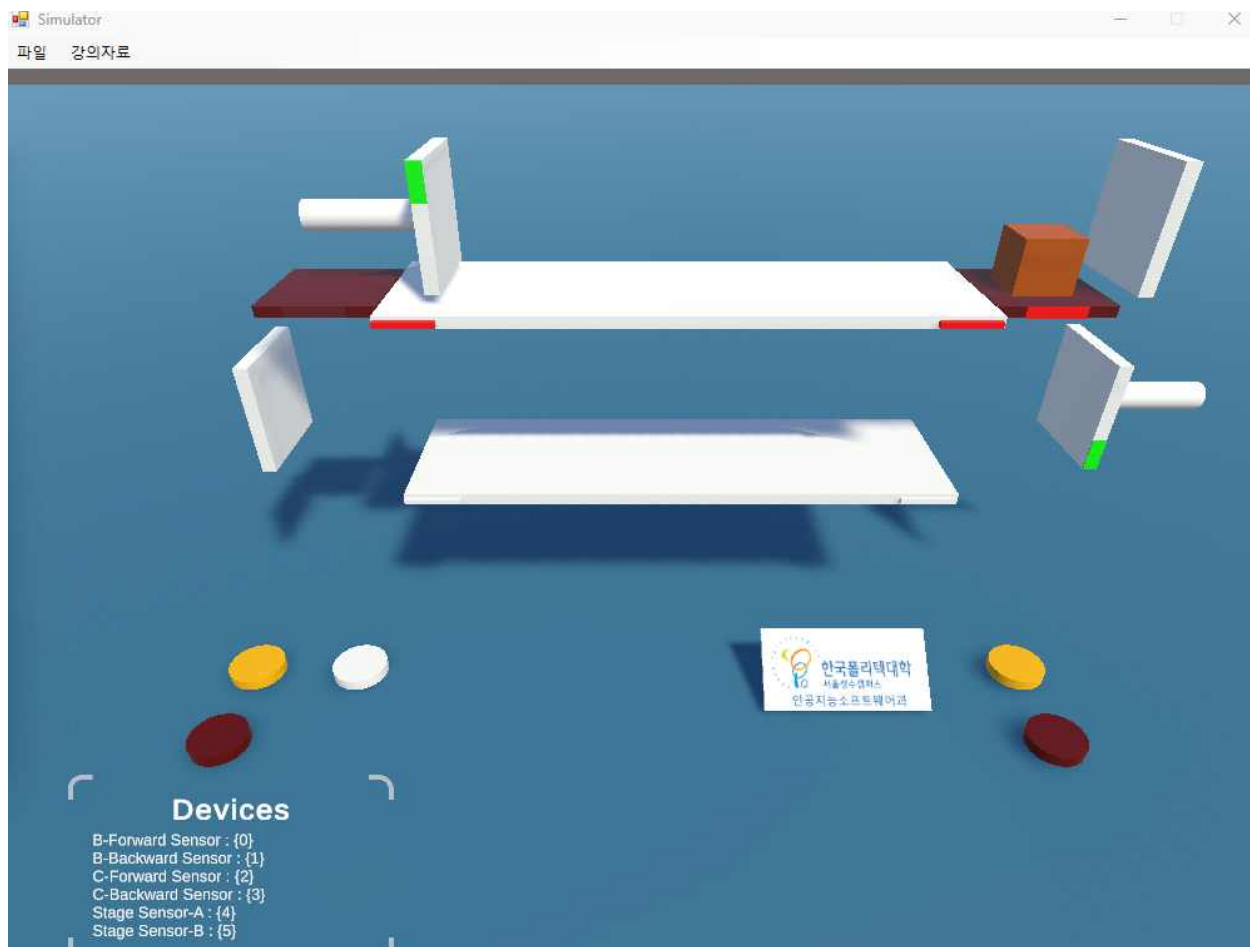


PLC 연결이 완료된 후 프로그램이 수동 대기 모드로 전환된 화면입니다. 자동 운전을 시작하기 전이므로 B·C실린더와 리프트A·B의 수동 조작 버튼이 활성화되어 있습니다.

화면에는 B실린더와 C실린더가 후진 완료 상태로 표시되었으며, 리프트A는 상승 완료, 리프트B는 하강 완료 상태로 확인되었습니다. 또한 리프트센서 A는 ON, 리프트센서 B는 OFF로 표시되어 현재 물체의 위치를 GUI에서 확인할 수 있었습니다.

시뮬레이터에서도 물체와 각 장치가 초기 위치에 배치된 것을 확인하였습니다. 이를 통해 PLC 연결 이후 센서값과 장치 상태가 GUI에 정상적으로 표시되는 것을 검증하였습니다.

2) 자동 운전 시작 및 B실린더 전진

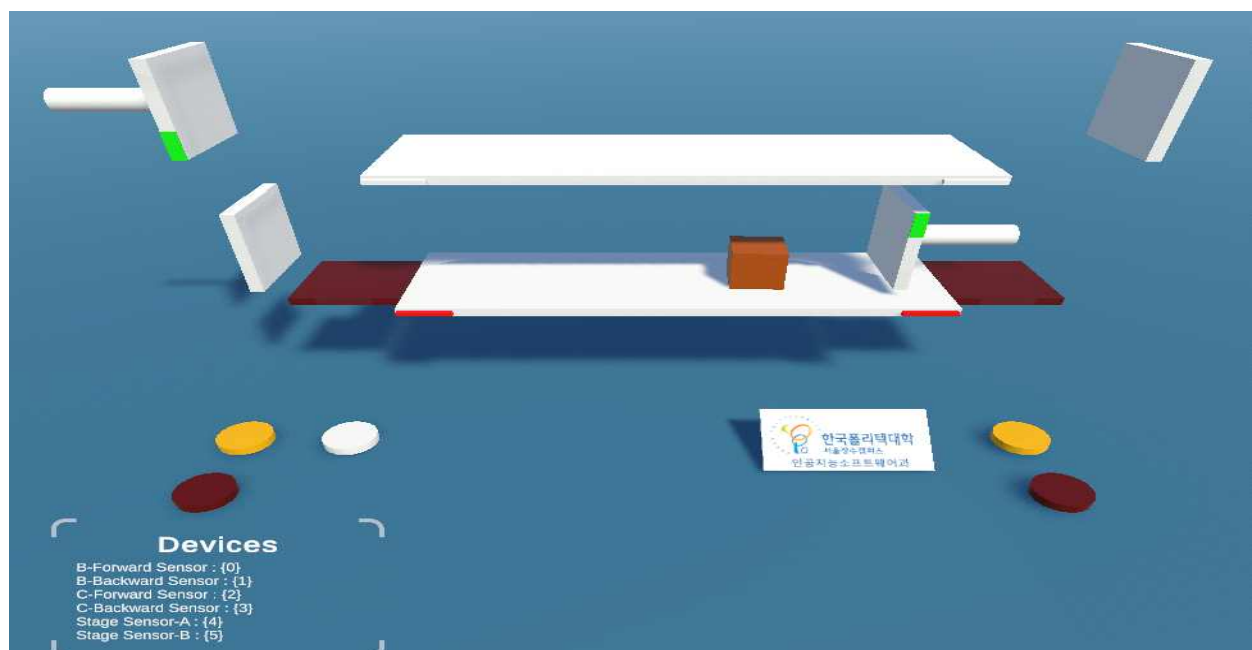


자동시작 버튼을 누른 후 B실린더가 물체를 리프트B 방향으로 이동시키는 화면입니다. 화면 상단에는 자동: B실린더 전진(센서B 감지 대기) 문구가 표시되어 현재 자동 운전 단계를 확인할 수 있습니다.

자동 운전 중에는 수동 조작 버튼이 비활성화되어 사용자가 실린더나 리프트를 직접 조작할 수 없도록 하였습니다. B실린더는 전진 완료 상태로 표시되었으며, 프로그램은 리프트센서 B가 물체를 감지할 때까지 기다리는 상태입니다.

이를 통해 자동 운전이 시작되면 수동 명령이 차단되고, 현재 자동 단계가 GUI에 정상적으로 표시되는 것을 확인하였습니다.

3) C실린더 전진 및 센서A 감지 대기



리프트B가 하강한 후 C실린더가 물체를 리프트A 방향으로 이동시키는 화면입니다. 화면 상단에는 자동: C실린더 전진(센서A 감지 대기) 문구가 표시되었습니다.

이때 B실린더는 후진 완료 상태이고, C실린더는 동작 중으로 표시되었습니다. 리프트A와 리프트B는 모두 하강 완료 상태이며, 리프트센서 B는 ON으로 확인되었습니다.

시뮬레이터에서는 물체가 아래쪽 이동 구간에 놓여 있고, C실린더가 물체를 리프트A 방향으로 밀어 이동시키는 모습을 확인할 수 있었습니다. 이를 통해 이전 단계가 완료된 후 C실린더 전진 단계로 정상적으로 전환되는 것을 검증하였습니다.

4) 리프트A 상승 단계



C실린더가 물체를 리프트A까지 이동시킨 후 리프트A가 상승하는 화면입니다. 화면 상단에는 자동: 리프트A 상승이 표시되었으며, 리프트A 상태는 동작 중으로 나타났습니다. 리프트센서 A가 ON으로 표시되어 물체가 리프트A 위치에 도착한 것을 확인할 수 있었습니다. 또한 C실린더는 물체 이송을 완료한 후 후진 완료 상태로 복귀하였습니다. 시뮬레이터에서도 물체가 리프트A 위에 놓인 상태로 상승하는 것을 확인하였습니다. 이 단계가 완료되면 프로그램은 다시 리프트B 준비 단계로 이동하여 다음 자동 운전 주기를 시작합니다.

5) 자동 운전 정지 및 수동 모드 복귀

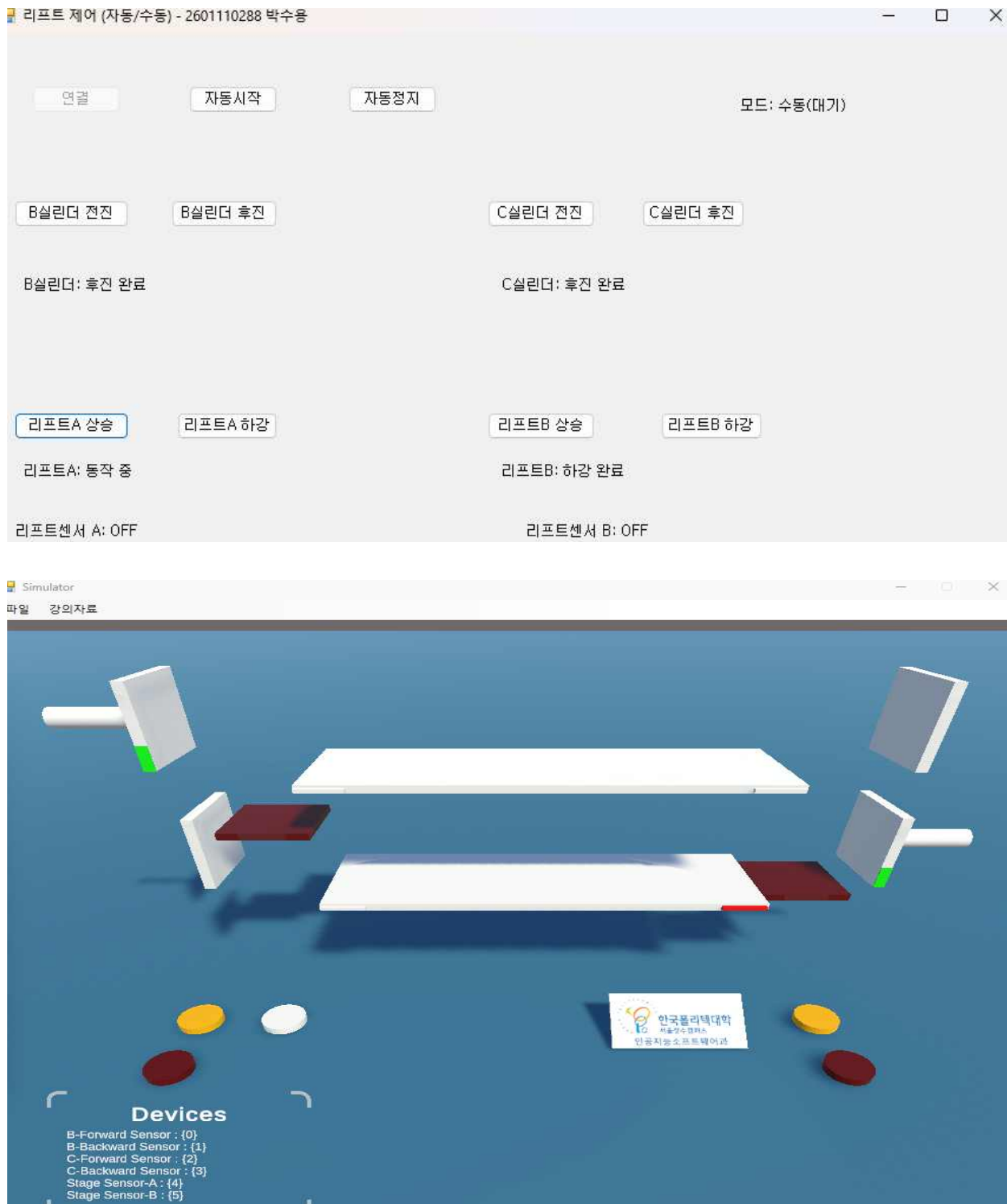


자동정지 버튼을 눌러 자동 운전을 중지한 화면입니다. 정지 후 화면 상단의 모드가 수동(대기)로 변경되었으며, 자동 운전 중 비활성화되어 있던 수동 조작 버튼이 다시 활성화되었습니다.

B실린더와 C실린더는 후진 완료 상태로 표시되었으며, 리프트A와 리프트B는 모두 상승 완료 상태로 확인되었습니다. 또한 리프트센서 A는 OFF, 리프트센서 B는 ON으로 표시되어 자동 운전을 정지한 시점의 물체 위치가 GUI에 반영된 것을 확인하였습니다.

이를 통해 자동정지 명령이 실행되면 자동 시퀀스가 중단되고, 사용자가 다시 각 컴포넌트를 수동으로 조작할 수 있는 상태로 복귀하는 것을 확인하였습니다.

6) 수동 모드에서 리프트A 개별 조작



수동 모드에서 리프트A 상승 버튼을 직접 클릭한 화면입니다. 리프트A 상태가 동작 중으로 변경되었으며, 시뮬레이터에서도 리프트A가 상승 방향으로 움직이는 것을 확인하였습니다. 리프트A 상승 출력이 정상적으로 적용되었으며, 같은 리프트의 하강 출력은 동시에 해제되는 것을 확인하였습니다. 또한 이번 검증에서는 다른 수동 버튼을 동시에 누르지 않고 리프트A 상승 기능만 단독으로 실행하였습니다.

같은 방법으로 B·C실린더의 전진과 후진, 리프트A·B의 상승과 하강 버튼을 각각 실행하여 총

8개의 수동 조작 기능이 정상적으로 동작하는 것을 검증하였습니다.

위 실행 화면을 통해 PLC 연결, 자동 운전 시작, 센서 조건에 따른 단계 전환, 자동 정지, 수동 개별 조작 기능이 정상적으로 수행되는 것을 확인하였습니다. 또한 GUI에 현재 운전 모드와 각 컴포넌트의 상태가 실시간으로 표시되어 사용자가 전체 동작 상황을 확인할 수 있었습니다.

7. 구현 결과 및 자체평가

7.1 한계점과 개선 방향

현재 프로그램은 과제에서 요구한 기능을 수행하지만, 실제 장비에 적용하기 위해서는 다음과 같은 보완이 필요합니다. 기능을 과장하기보다 현재 코드에서 확인되는 한계를 기준으로 정리하였습니다.

현재 한계	문제가 되는 이유	개선 방향
예외 내용을 기록하지 않음	빈 catch문만 사용하면 통신 오류 원인을 확인하기 어려움	오류 코드와 발생 시간을 로그 파일 또는 화면에 기록
컨트롤 이름이 button1, label1 형태	코드만 보고 버튼 역할을 바로 구분하기 어려움	btnAutoStart, lblLiftA 등 의미 있는 이름으로 변경
통신·제어·화면 코드가 Form1에 집중	기능이 늘어날수록 수정 범위가 커짐	PLC 통신 클래스와 자동 제어 클래스로 분리
재시도 안내 문구가 잠시 남을 수 있음	통신이 복구되어도 모드 문구가 바로 복원되지 않을 수 있음	통신 성공 시 현재 모드·단계 문구를 다시 표시

7.2 프로젝트를 통해 배운 점

이번 과제를 통해 UI 버튼 이벤트만 작성하는 것과 실제 제어 프로그램을 구성하는 것은 다르다는 점을 확인하였습니다. PLC 입력과 출력은 비트 단위로 정확하게 매핑해야 하며, 자동 운전은 명령을 보내는 것에서 끝나는 것이 아니라 완료 센서를 확인한 뒤 다음 단계로 넘어가야 안정적으로 동작하였습니다.

또한 사용자가 잘못된 버튼을 누르지 못하게 하는 UI 처리, 통신 실패 시 프로그램을 유지하는 예외 처리, 종료 시 출력을 끄는 정리 과정도 제어 기능만큼 중요하였습니다. 특히 문서와 실제 시뮬레이터의 차이를 직접 확인하고 수정한 경험이 이번 과제에서 가장 의미 있는 부분이었습니다.

8. 결론

이번 프로젝트에서는 C# Windows Forms와 PLC 시뮬레이터를 연결하여 리프트 제어 시스템을 구현하였습니다. 자동 운전은 상태 머신을 사용해 실린더와 리프트의 동작을 센서 조건에 따라 순서대로 진행하였고, 수동 모드는 8개의 버튼으로 각 컴포넌트를 개별 제어할 수 있도록 구성하였습니다. 또한 타이머를 이용해 센서 상태를 주기적으로 읽고 GUI에 표시하였습니다.

구현 과정에서는 입출력 문서와 실제 시뮬레이터 반응이 다른 문제를 직접 확인하여 매핑을 수정하였고, 타이머 재진입 방지와 중복 출력 감소를 적용해 통신 안정성을 높였습니다. 최종적으로 과제에서 요구한 자동 운전, 수동 조작, 상태 표시, 코드 설명, 시나리오 검증을 모두 수행하였습니다.

추후에는 컨트롤 이름을 의미 있게 변경하고, 통신 오류 로그를 추가하며, PLC 통신과 자동 제어 로직을 별도 클래스로 분리하고자 합니다. 이를 통해 현재 기능을 유지하면서도 코드를 더 쉽게 수정하고 확장할 수 있을 것으로 판단하였습니다.